

Azure Local Core Technologies



Agenda

- **Objective:** Review Failover Clustering (FC) and S2D architecture for Azure Local high availability and software-defined storage.
- **Scope:** FC (HA, failover, networking, quorum, CSV), S2D (storage pooling, cache/tiering, resiliency, ReFS, SMB3/RDMA), performance/efficiency tradeoffs, volume planning.



Failover Clustering: What & Why

- Independent servers (nodes) that work together to keep clustered roles (apps/VMs/services) running during hardware or software failures.
 - Automatic failover: If node or role health degrades, the cluster restarts the role or moves it to a healthy node.
- Continuous health monitoring:
 - Heartbeats assess node connectivity and role responsiveness; cluster acts when signals fail.
- Outcome: Minimized downtime, continuous availability of Hyper-V VMs, file services, SQL, and other mission-critical workloads.

Cluster Networking: Private vs. Public

- Separating internal traffic from client traffic for performance & safety
- Private networks (intra-cluster):
 - Heartbeat & management traffic (node-to-node health, coordination).
 - Storage coordination (S2D Software Storage Bus, CSV I/O redirection when needed).
- Public networks (client-facing):
 - Application data paths (VM access, SMB client connections, SQL traffic).
- Benefits of separation:
 - Performance stability (cluster control traffic insulated from client spikes).
 - Fault isolation & security (reduce exposure of critical cluster functions).
 - High availability preserved during external congestion.

Quorum & Witness: Preventing Split-Brain

- **Quorum prevent split-brain (two node subsets acting as owners):**
 - Cluster remains online only when majority of votes are present.
- **Quorum models:**
 - Node Majority (each node has a vote).
 - File Share Majority / Cloud Witness (extra witness vote to break ties).
 - Disk Witness (not supported with S2D).
- **Recommendations:**
 - 2 nodes: Witness is required.
 - 3–4 nodes: Witness strongly recommended.
 - ≥ 5 nodes: Witness not needed (no added resiliency).
- **Dynamic quorum & dynamic witness:**
 - Automatically zeroes a node vote or toggles witness vote to maintain odd total and keep cluster online.

Active-Active vs. Active-Passive

- Selecting an operation mode matching availability & utilization goals
- **Active-Active (all nodes serve workloads):**
 - Load balancing distributes requests; no idle hardware.
 - Better throughput and resource efficiency; capacity dips gracefully on single-node loss.
- **Active-Passive (standby nodes):**
 - Idle nodes ready to take over on failure—simpler to manage.
 - Predictable failover behavior; hardware underutilized in steady state.
- **Choose based on priority:**
 - Performance & utilization (Active-Active) vs. simplicity & determinism (Active-Passive).

Cluster Shared Volumes (CSV)

- Shared, unified storage namespace for concurrent access
- **What CSV provides:**
 - Single namespace (e.g., C:\ClusterStorage\) visible on all nodes.
 - Concurrent access to shared volumes for VMs and applications.
- **Operational behavior:**
 - Transparent failover: volumes appear locally mounted everywhere; I/O continues as roles move.
 - Supports S2D, Hyper-V, and Scale-Out File Server (SoFS) scenarios.
- **Benefit:**
 - Simplified management and seamless mobility for stateful workloads.

Storage Spaces Direct (S2D): Overview

- **Turning direct-attached drives into a high-performance, resilient pool**
 - Software-defined storage aggregating local drives across 2–16 servers into a single pool with cache, tiers, and resiliency.
- **How S2D works (at a glance):**
 - Software Storage Bus forms a cluster-wide storage fabric: all nodes see each other's drives (no Fibre Channel/shared SAS needed).
 - CSV + ReFS: Volumes created on the pool run CSV with ReFS for fast VHDX ops and integrity.
 - SMB3/RDMA networking: East-west I/O uses SMB Direct for low latency and CPU efficiency.
- **Deployment forms:**
 - **Hyperconverged** (VMs on the same cluster that provides storage)
 - **Converged/SoFS** (storage cluster + compute cluster).

How Storage Spaces Direct Works

- **Local drives pooled into unified storage:**
 - All direct-attached SATA/SAS/NVMe/PMem disks are automatically discovered and added to a single storage pool.
 - Pool supports cache, tiers, resiliency, and intelligent drive onboarding.
- **Software Storage Bus provides cluster-wide visibility:**
 - Creates a virtual fabric across nodes.
 - Every server sees every other server's drives; no SAN/Fibre Channel needed.
- **Volumes created on the pool:**
 - Volumes use CSVFS + ReFS for optimized virtualization performance.
 - Appear on all nodes under C:\ClusterStorage\.
- **Built-in orchestration:**
 - Self-healing, automatic rebalancing, and continuous assessment of drive/node health.

S2D Architecture

- **Networking:** SMB3, SMB Direct, RDMA (iWARP or RoCE); 10+ GbE recommended for low-latency east-west traffic.
- **Storage Hardware:** NVMe/SSD/HDD/PMem; minimum 2 cache-capable drives + ≥ 4 capacity per node; direct-attached only.
- **Software Storage Bus:** Replaces external fabrics; each server sees cluster drives; coordinates resiliency & placement.
- **Storage Bus Layer Cache (SBL):** Binds fastest drives (e.g., NVMe) to slower media (SSD/HDD) for read/write acceleration.
- **ReFS + CSV:** Integrity + performance (checksums, fast VHDX, namespace uniformity).
- **Scale-Out File Server (SoFS):** Converged scenario: exposes SMB3 shares from S2D to external clients/compute.

S2D Storage

- **Supported drive types:**
 - NVMe (PCIe-attached SSDs: U.2, AIC, M.2) - high IOPS/low latency
 - SSD (SATA/SAS)
 - HDD (large-capacity spinning disks)
 - PMem (persistent memory) for ultra-low latency
- **Drive requirements:**
 - Each node: ≥ 2 SSD/NVMe for cache, ≥ 4 capacity drives.
 - Drives must be direct-attached (no shared SAS).
 - Up to 16 nodes and 400+ drives per cluster.
- **Automatic tiering:**
 - Fast drives automatically assigned to cache tier; slower ones become capacity.

S2D Networking

- **Protocol foundation:**
 - SMB3, SMB Direct, RDMA (iWARP or RoCE) are used for all east-west storage traffic.
- **Performance expectations:**
 - 10+ GbE strongly recommended; many deployments use 25/40/100 GbE.
 - RDMA dramatically reduces CPU load during storage operations.
- **Why it matters:**
 - S2D depends on constant synchronization between nodes.
 - Slow or misconfigured networks cause latency, timeouts, and degraded resiliency behavior.

The Software Storage Bus

- **Cluster-wide storage fabric:**
 - All nodes participate in a distributed software bus; all drives are visible to all nodes.
- **Replaces expensive storage fabrics:**
 - Eliminates Fibre Channel, shared SAS, or storage arrays.
- **Drives are virtualized into the pool:**
 - Placement, resiliency calculations, and cache bindings occur at the bus layer.

Storage Bus Layer Cache (SBL Cache)

- **Automatic cache selection:**
 - Fastest drives = **cache**; others = **capacity**.
- **Behavior depends on media:**
 - All-flash (NVMe → SSD): **write-only** caching
 - Hybrid (SSD → HDD): **read + write** caching
- **Purpose:**
 - Reduce latency, coalesce writes, derandomize HDD I/O, and extend SSD endurance.

ReFS: The Filesystem for Virtualization

- **Designed for virtualization:**
 - Extremely fast VHDX creation, checkpoint merging, and block cloning.
- **Built-in protection:**
 - Checksums, integrity streams, and automatic corruption repair with S2D redundancy.
- **Real-time tiering:**
 - Moves “hot” data to fast media; cold data to slower tiers automatically.

Cluster Shared Volumes (CSV) in S2D

- **Single, unified namespace:**
 - All volumes appear under C:\ClusterStorage\ on all nodes.
- **Concurrent multi-node access:**
 - Enables Hyper-V, SoFS, and other clustered workloads.
- **Volume mobility:**
 - Volumes remain accessible during node failover; I/O redirects as needed.

S2D Volume Types

- **Volumes = Virtual Disks:**
 - Defined on the storage pool, exposed via CSV, formatted as ReFS.
- **Choice of resiliency models:**
 - Two-way mirror (2 copies)
 - Three-way mirror (3 copies)
 - Dual parity (Reed-Solomon / LRC)
 - Mirror-accelerated parity (hybrid)
- **Decoupled from hardware:**
 - Resiliency types do not depend on specific drive types.

Understanding Fault Tolerance in S2D

- **Resiliency similar to RAID**, but **distributed across servers** instead of drives in a single chassis.
- **Two categories of resiliency:**
 - **Mirroring** (multiple full data copies).
 - **Parity/Erasure Coding** (mathematical reconstruction).
- **Survivable hardware failures include:**
 - Loss of a drive, loss of a server, drive + server, or multiple drives across different servers.
- **Key constraint:** S2D can tolerate **up to two simultaneous server faults** (depending on resiliency).
- **Self-healing engine** uses surviving slabs to repair/restore lost data.

S2D Resiliency Options

- **Mirroring (fastest I/O, lower efficiency):**
 - Two-way mirror (≥ 2 servers): 2 copies, ~50% efficiency, tolerates 1 failure.
 - Three-way mirror (≥ 3 servers): 3 copies, ~33% efficiency, tolerates 2 failures.
 - Best for performance-sensitive workloads (SQL, busy Hyper-V).
- **Parity (higher efficiency, higher write latency):**
 - Dual parity (≥ 4 servers): ~50–80% efficiency depending on scale; tolerates 2 failures.
 - Best for sequential or cold workloads (backup/archival, VDI at scale).
- **Mirror-Accelerated Parity (hybrid):**
 - Writes land in mirror, later destage to parity; balance ingest performance and capacity.
 - Size mirror portion to accommodate peak write bursts.
- **Nested resiliency (2 node-clusters only)**

Nested Resiliency

- Designed for 2-node clusters only.
- Two nested resiliency modes:
 - Nested two-way mirror (four total copies).
 - Nested mirror-accelerated parity (mirror + parity structure).
- **Survivability:**
 - Survives two hardware failures (e.g., server + drive, two drives).
- **Trade-off:**
 - Lower capacity efficiency (25–40%).

Volume Resiliency: Choosing the Right Type

- Performance vs. capacity vs. fault tolerance
- **When performance matters:**
 - Choose mirroring (2-way or 3-way).
 - Best for SQL, high-IOPS VMs, random write workloads.
- **When capacity matters:**
 - Choose dual parity.
 - Best for archive, cold storage, large sequential writes.
- **When ingestion speed matters:**
 - Use mirror-accelerated parity.
 - Writes land in mirror, destage to parity.
- **Mix & match:**
 - Different volumes can use different resiliency models in the same cluster.

Mirroring: Two-Way vs Three-Way

- High performance / lower efficiency resiliency models
- **Two-Way Mirror**
 - Writes two copies across two servers.
 - 50% storage efficiency (2 TB footprint for 1 TB usable).
 - Survives one hardware fault.
- **Three-Way Mirror**
 - Writes three copies across three servers.
 - 33.3% storage efficiency.
 - Survives two simultaneous faults.
- **When to choose mirroring:**
 - Highly random I/O, VMs, SQL, critical low-latency workloads.
 - When predictable performance is required.

Parity (Erasure Coding)

- **Better Capacity Efficiency, Higher Write Latency**
- **Single Parity (RAID-5-like):**
 - Stores 1 parity symbol; tolerates one failure.
 - Requires ≥ 3 servers.
 - Not recommended (three-way mirror is safer at this scale).
- **Dual Parity (RAID-6-like / Reed-Solomon):**
 - Stores 2 parity symbols, tolerates two failures.
 - Requires ≥ 4 servers.
 - 50–80% efficiency depending on node count.
- **Trade-offs:**
 - Higher CPU use, higher latency, especially during random writes.
 - Ideal for large sequential writes and cold storage workloads.

Local Reconstruction Codes (LRC)

- Optimized Parity for Large Clusters
- Advanced erasure coding technique used inside **dual parity** at larger scales.
- Splits **parity groups** into **smaller clusters** to reduce CPU overhead.
- Improves write throughput and rebuild speed.
- Example: **8 data + 2 parity + 1 local parity group** in large SSD deployments.

Mirror-Accelerated Parity (MAP)

- Hybrid model for high ingest + good capacity savings
- **Two-layer volume structure:**
 - Mirror portion (fast landing zone for writes).
 - Parity portion (capacity-efficient final storage).
- **Write process:**
 - Writes hit **mirror** first → later **destaged** to parity.
- **Best for workloads with large sequential write bursts:**
 - Backup ingestion, archival pipelines, periodic bulk writes.

Pool Quorum

- Ensure the storage pool remains online independently of cluster quorum
- Pool quorum vs. cluster quorum:
 - Cluster quorum: determines if the cluster stays online.
 - Pool quorum: determines if the storage pool stays online.
- Pool uses drive votes:
 - Each metadata-bearing drive gets a **vote**.
 - Must maintain **50% + 1** of votes to stay online.
- Differences from cluster quorum:
 - No dynamic quorum.
 - No **vote removal** mechanism.
 - Uses **cluster DB** to break ties.

When Pools Stay Online vs. Offline

- **Pool stays online when:**
 - Losing **one node**, losing **one drive**, or losing **two servers** (with three-way mirror/dual parity).
 - Example: 4-node symmetric pool survives 2-node loss (9/16 drive votes remain).
- **Pool goes offline when:**
 - Too many nodes/drives fail → below voting majority.
 - Example: drive failures + multiple node failures reducing metadata votes.

Designing with Pool Quorum in Mind

- **Ensure symmetrical drive distribution:**
 - All nodes should have the same number of drives.
- **Prefer three-way mirror or dual parity for multi-fault tolerance.**
- **Maintain reserve capacity:**
 - Keep 1 capacity drive per server unallocated (up to 4).
- **Replace failed drives promptly to maintain quorum safety margins.**

Choosing Drives for S2D

- Performance, Endurance, and Capacity Planning
- Supported drive types:
 - NVMe (highest performance), SSD, HDD
- Caching recommendations:
 - Fastest drives (e.g., NVMe) → cache
 - SSD/HDD → capacity
- Endurance considerations:
 - Use high-endurance NVMe/SSD for caching to maximize write lifetime.
- Hybrid planning:
 - NVMe + HDD for balanced cost/performance.
 - SSD + HDD for capacity-first designs.

Drive Caching Behavior: All-Flash vs Hybrid

- **All-Flash deployments (NVMe → SSD):**
 - Cache operates in **write-only** mode.
 - Reads come **directly** from SSD (latency already low).
- **Hybrid deployments (SSD → HDD):**
 - Cache provides **both read and write** caching.
 - Read cache holds **frequently/recently accessed data**.
 - Write cache **coalesces writes** and **derandomizes** for HDD throughput.
- **Cache drives never contribute usable capacity.**
 - They store transient data; all persisted blocks reside on capacity drives.

Dynamic Cache Binding

- **S2D Reassigns Cache-to-Capacity Relationships Automatically**
- **Flexible binding ratios:**
 - Ranges from 1:1 to 1:12+ (cache:capacity).
- **Automatic rebinding events:**
 - When drives are **added**, **removed**, or **fail**, cache bindings recalculate automatically.
- **Why this matters:**
 - Enables **independent scaling** of cache and capacity tiers.
 - Maintains balanced performance even in large or growing clusters.
- **Best practice:**
 - Maintain **symmetrical drive counts** for smoother rebalance behavior.

Cache Drive Failure Handling

- What Happens When an NVMe/SSD Cache Drive Fails
- Impact on data:
 - Cached writes exist **temporarily**; persistent data is always stored redundantly on capacity drives.
- Behavior on failure:
 - Capacity drives bound to the failed cache drive appear “**degraded**.”
 - S2D **rebinding** moves their cache workloads to surviving cache drives.
- After rebinding:
 - System automatically triggers **repair jobs** using existing mirrored/parity data.
 - Performance returns to normal after repair completes.

CSV Cache (In-Memory Read Cache)

- **CSVFS read cache:**
 - RAM-based, accelerates read-heavy workloads.
 - Independent from the SBL cache.
- **Enabled by default** on Azure Local.
- **Works well when:**
 - VMs repeatedly access the **same blocks**.
 - Data is mostly **read-hot** but not write-intensive.

Manual Cache Configuration (Advanced)

- Overriding automatic cache selection when needed
- Why override?
 - When cluster uses **all-flash** (all NVMe or all SSD) with mixed **endurance** grades.
- Administrator can specify:
 - Which drive model acts as **cache** using:
 - `Enable-ClusterS2D -CacheDeviceModel "MODEL"`
- Validation:
 - Use Get-PhysicalDisk → verify Usage = "Journal".
- When to avoid:
 - Unless you have mixed device endurance, the **automatic cache selection** is preferable.

Volume Planning: Count & Size

- **Recommended limits:**
 - ≤ 64 volumes per cluster.
 - Minimum: one volume per node for balanced ownership.
- **Volume sizing:**
 - Recommended ≤ 64 TB per volume (Azure Local).
 - For VSS-based backup systems, keep to ≤ 10 TB for better stability.
- **File system choices:**
 - Prefer ReFS; use NTFS only when required by specific features.

Volume Allocation Methods

- **Regular allocation (default):**
 - Volume slabs **distributed evenly across all servers**.
 - Maximizes performance, parallelism, and simplicity.
- **Delimited allocation (advanced):**
 - Volume restricted to **subset of servers**.
 - Useful when requiring **survival of 3 simultaneous failures**.
 - Minimum **4 nodes** required.
- **Trade-off:**
 - More fault tolerance vs. **increased management complexity**.

Why Use Delimited Allocation?

- Enhanced failure survivability beyond default S2D behavior
- Survive three simultaneous node/drives domain failures (in some patterns).
- Useful for:
 - Clusters operating with non-uniform server topologies,
 - Deployments requiring higher fault domain separation,
 - Scenarios where certain nodes are more failure-prone.
- Not recommended for small clusters (<6 nodes).

How to Create Delimited Volumes

- Step 1: Identify servers:
 - `Get-StorageFaultDomain -Type StorageScaleUnit`
- Step 2: Create volume with subset:
 - `New-Volume -StorageFaultDomainsToUse $Servers[0,1,2,3] -Size 100GB`
- Step 3: Validate placement:
 - Use `Get-VirtualDiskFootprintBySSU` script.
- Step 4: Rebalance storage pool:
 - Run `Optimize-StoragePool` after changes.

Moving a Delimited Volume

- Adjusting StorageFaultDomainsToUse after server replacement or expansion
- Add Fault Domain:
 - `Add-StorageFaultDomain -StorageFaultDomains $Servers[4]`
- Remove older domain:
 - `Remove-StorageFaultDomain -StorageFaultDomains $Servers[0]`
- Rebalance the pool:
 - `Optimize-StoragePool` afterwards.
- Verify changes:
 - Check footprint again with `Get-VirtualDiskFootprintBySSU.`

cloudops *gility*